

HuggingFace Weekly Papers Finder

RAG 기반 최신 ML/DL/LLM 논문 검색 챗봇

SKN네트웍스 Family AI 캠퍼스 20기

Project 3rd - 2 TEAM

팀 : 해조

목차

1. RAG 진행과정 정리
2. 좋은 RAG 시스템을 만드는 7단계 전략
3. PROCESS 소개
4. 성능 비교
5. 최종 시스템 흐름 다이어그램

1. RAG 진행과정 정리

- 1 - 검색 : 관련 문서 찾아오기
- 2 - 필터링 : 중요한 문서 선별
- 3 - 청킹 : 문서를 의미 단위로 분리
- 4 - 컨텍스트 주입 : LLM에 문서 삽입
- 5 - 생성 : 문서 기반 신뢰 답변 생성

2. 좋은 RAG 시스템을 만드는 전략

**'환각을 최소화하고, 정확하고 최신·도메인 특화 정보를
활용한 신뢰성 높은 AI'를 만드는 것**

- 그중에서 프로젝트 주제에 맞게 질의응답(Q&A) RAG시스템을 다룰 예정

3. PROCESS 소개

- 1) 데이터 전처리**
- 2) 랭 시스템 / 랭그래프 시스템**
- 3) html 구현**

0) 주제 선정



HuggingFace Weekly Papers Finder

RAG 기반 최신 ML/DL/LLM 논문 검색 챗봇

22개 후보 중 팀원 다중선택 투표로 최종 주제 선정

1) 데이터 전처리 과정

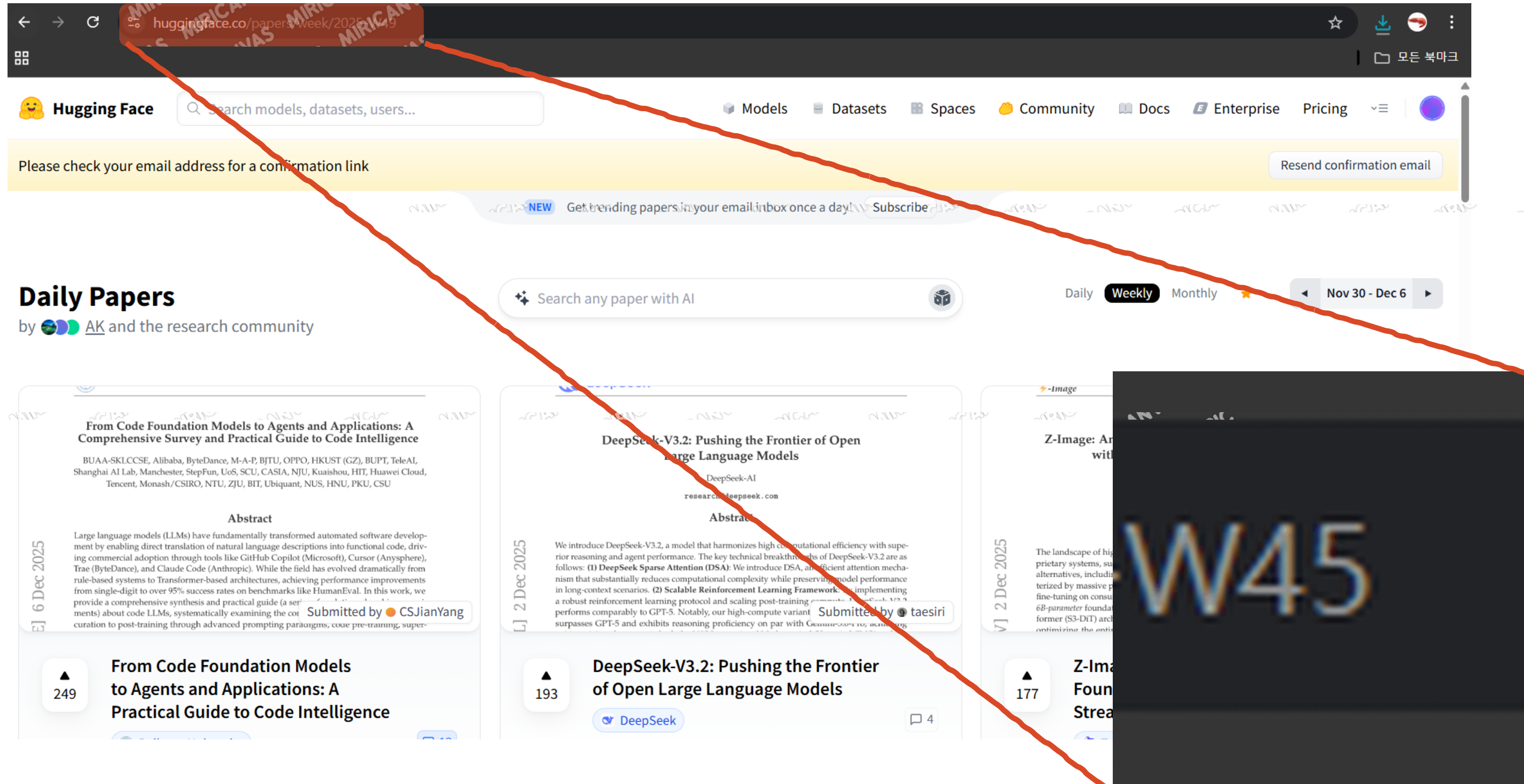
크롤링

청킹

클러스터링

벡터DB

1) 데이터 전처리 - 크롤링



크롤링 45~49

1) 데이터 전처리 - 청킹

청킹 전략

청킹 전략 목적

문서를 일정 단위로 나누어 검색 정밀도 향상

LLM이 처리할 수 있도록 문맥 단위 유지

청크 하이퍼파라미터 조정으로 RAG 품질 최적화

(Chunk Size, Chunk Overlap)

1) 데이터 전처리 - 청킹

청킹 하이퍼파라미터 조정

1. Chunk Size 후보 선정 과정

문서 평균 길이 분석 결과:

→ 약 300 Token 부근이 문서 1단위에 해당

실험을 위해 300 / 200 / 100 선정

1) 데이터 전처리 - 청킹

청킹 하이퍼파라미터 조정

2. Chunk Overlap 후보 선정 과정
연구 기반 오버랩 기준

여러 연구 결과:

→ 전체 청크 길이의 10~25% 오버랩이 가장 효율적

본 프로젝트에서는 중간값($\approx 15\%$)을 기준으로 선정

1) 데이터 전처리 - 청킹

청킹 하이퍼파라미터 조정

Chunk Size별 오버랩 설정

Chunk Size	15% Overlap	No Overlap(0)
300	45	0
200	30	0
100	15	0

오버랩 후보에 '0' 을 포함한 이유 :

일부 연구(특히 화학 논문 QA 벤치마크)에서
→ 오버랩 0일 때 성능 향상 사례 발견

따라서

비교 실험을 위해 No-Overlap 조합도 추가



Chunk Size × Chunk Overlap 적용 조합 (총 6가지)

Chunk Size	Overlap	설명
300	45	문맥 보존 강화
300	0	무오버랩, 독립 청크 유지
200	30	중간 밀도 문맥 유지
200	0	무오버랩, 빠른 검색
100	15	고세분화 + 약간의 문맥 보완
100	0	초고세분화, 완전 독립 처리

1) 데이터 전처리 - 키워드 추출

초기 접근 방식

초기 검색 방식

TF-IDF, KeyBART 기반 검색 적용

문제 발견

SimpleRAGSystem2 테스트 결과

→ 관련 문서를 제대로 찾지 못함

1) 데이터 전처리 - 키워드 추출

오류 모색 : 유니크한 태그갯수 조회

```
총 JSON 파일 개수 : 506
```

```
총 태그 개수 : 1518
```

```
고유 태그 개수 : 1445
```

```
=====
```

```
각 태그별 빈도:
```

```
=====
```

```
language models: 15 | large language: 9 | multimodal large: 6
```

```
language action: 4 | video generation: 4 | learning rl: 4 | que
```

```
human annotations: 3 | generative: 2 | video motion: 2 | reward
```

```
multimodal benchmarks: 2 | image t2i: 2 | policy optimization:
```

1) 데이터 전처리 - 키워드 추출

염려점

유니크 태그 개수 조회

예상보다 태그 개수가 너무 많음 → 1,445개

희귀 태그 다수 → 검색 품질 저하 가능성

* why 문제인가?

- 랭킹 불안정
- 노이즈 증가
- 색인 구조 비대화 → 검색 속도 저하
- DB 공간 사용량 증가
- 태그-문서 매핑 테이블 관리 비용 상승

⇒ 희귀 태그가 불필요하게 많으면 여러모로 안좋다

1) 데이터 전처리 - 키워드 추출

개선 방안 (1) : 클러스터링 적용

후에 오류 수정 방안 (2) : BM25으로 변경

태그를 직접 추출 X

- 유사한 태그들을 클러스터링하여 묶음 단위로 처리
- 클러스터링 탐색시 답변 정확도가 떨어질 수 있음

목적 :

- 태그 노이즈 감소
- 검색 안정성 향상
- 전체 문서를 더 큰 의미 단위로 이해

1) 데이터 전처리 - 키워드 추출

방법 :

OpenAI Embedding Model 활용하여 전체 태그를 임베딩 → 클러스터링 수행

K-Means를 사용하여 유사한 태그들이 모인 뭉치들을 만들어서
전체 문서를 읽게하여 10~30개 사이의 클러스터를 생성
→ 5개의 태그가 들어가는 18개의 클러스터

! 변경사항 :

크롤링 40~49 로 변경 - 클러스터링을 하기위해 더 많은 데이터가 필요해서, 데이터량을 늘림

1) 데이터 전처리 - 키워드 추출

전체 키워드 통계 (빈도순)

총 키워드 수: 90
고유 키워드 수: 50

llm	: 7	reasoning	: 6	training	: 6	image	: 5	generation	: 5
multimodal	: 4	performance	: 3	video	: 3	world	: 3	language	: 3
token	: 2	scene	: 2	agent	: 2	quality	: 2	framework	: 2
feature	: 1	high	: 1	http	: 1	github	: 1	visual	: 1
motion	: 1	research	: 1	tool	: 1	knowledge	: 1	bench	: 1
long	: 1	diffusion	: 1	prompt	: 1	policy	: 1	instruction	: 1
image editing	: 1	control	: 1	code	: 1	large	: 1	step	: 1
spatial	: 1	attention	: 1	context	: 1	vla	: 1	robot	: 1
real	: 1	system	: 1	environment	: 1	benchmark	: 1	scientific	: 1
grpo	: 1	reward	: 1	understanding	: 1	unified	: 1	problem	: 1

1) 데이터 전처리 - 벡터 임베딩

(1) 임베딩 모델 성능 평가

(2) 평가 벡터DB 임베딩

1) 데이터 전처리 - 벡터 임베딩

(1) 임베딩 모델 성능 평가 - 후보 모델, 평가 메트릭 선정

임베딩 7개 모델후보 선정

BGE-M3

MPNet

MiniLM-L6

MsMarco

OpenAI-small

Paraphrase-Multi

SPECTER

평가 메트릭 선정

Hit Rate@K

MRR

NDCG@K

Avg Time

1) 데이터 전처리 - 벡터 임베딩

(1) 임베딩 모델 성능 평가 - 평가 메트릭 간단 설명

✗ Hit Rate@K

상위 K개 중 최소 1개 정답 포함 비율
→ 클러스터로 테스트 쿼리를 만들고
전체 문서를 사용했기 때문의 의미가 퇴색

✓ NDCG@K

상위 K개 결과의 전체 랭킹 품질을 평가
(관련도 높은 문서가 위로 올수록 좋음)

✓ MRR

첫 관련 문서가 얼마나 상위에 있는지 평가
(정답을 얼마나 빨리 찾는가)

△ Avg Time

쿼리당 평균 검색 시간(시스템 응답 속도 지표)
→ 테스트 할 때는 CUDA를 사용했기 때문에
적합한 평가가 안 될 수도 있음
문서양이 늘어나면 중요

1) 데이터 전처리 - 벡터 임베딩

(1) 임베딩 모델 성능 평가

평가 방식 :

임베딩 모델 7개 × 청킹 조합 6개

총 조합 42가지를 앞서 정의한 3가지 성능 지표*를 기준으로 성능 비교

테스트 쿼리 구성 :

18개 클러스터 × 각 클러스터에서 키워드 3개 추출

→ 해당 키워드로 생성질문 생성

* MRR, NDCG@K, Avg Time

1) 데이터 전처리 - 벡터 임베딩

모델	청크 전략	HR@5	HR@10	MRR	NDCG@5	NDCG@10	Avg Time(s)
Paraphrase-Multi	200_30	1.000	1.000	0.857	0.783	0.780	0.08
OpenAI-small	300_45	1.000	1.000	0.857	0.813	0.778	0.54
Paraphrase-Multi	300_0	1.000	1.000	0.810	0.754	0.765	0.04
OpenAI-small	300_0	1.000	1.000	0.821	0.771	0.762	0.41
Paraphrase-Multi	300_45	1.000	1.000	0.845	0.754	0.761	0.04
Paraphrase-Multi	200_0	1.000	1.000	0.800	0.724	0.754	0.07
MPNet	200_30	1.000	1.000	0.857	0.715	0.750	0.08
MiniLM-L6	300_45	1.000	1.000	0.881	0.754	0.745	0.02

1) 데이터 전처리 - 벡터 임베딩

최종 선정된 모델 - 청킹 조합 :

1) Paraphrase-Multi + Chunk 200_30

2) OpenAI-small + Chunk 300_45

1) 데이터 전처리 - 벡터 임베딩

(2) 벡터 임베딩

임베딩 모델 평가 로직 과정

chunking

|

clustering

|

evaluate_embeddings - improved_text_queries

실사용시 로직 과정

(임베딩 모델 선정 이후)

chunking

|

clustering

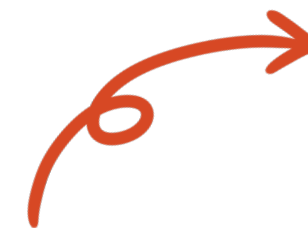
|

embedding

|

vectordb

임베딩 모델 선정 후 이 로직으로
VectorDB 저장



2) 랭체인 시스템

간단한 RAG CHAIN 시스템 흐름

랭체인 구조 :

벡터 DB에서 문서 검색

→ threshold를 기준으로 문서 필터

└ 문서 있으면 → 답변 생성

└ 문서 없으면 DuckDuckGo 웹 검색

→ 검색 결과 → 답변 생성

랭그래프 시스템 개발 시도 :

기존 시스템이 단순하고 확장성이 아쉬워서
서비스 품질을 확보하기 어려웠기 때문에,
더 고도화된 RAG 파이프라인으로 교체

2) 랭그래프 시스템

초반 랭그래프 구성

- + 하이브리드 검색기능 (BM25 + 유사도)
- + 클러스터 기반 조회 기능
- + tavily 웹 검색 기능 추가

초반 랭그래프 구성

STATE :

question:str

documents : List[Document]

doc_scores : List[float]

search_type : str

answer : str

NODE :

retriever 노드

grade 노드

generate 노드

랭 그래프 구조 :

START



Retriever (벡터DB 검색)



Grade Documents (관련성 평가)

└── generate → Generate (LLM 답변 생성)

└── web_search → Web Search (외부검색)



Generate



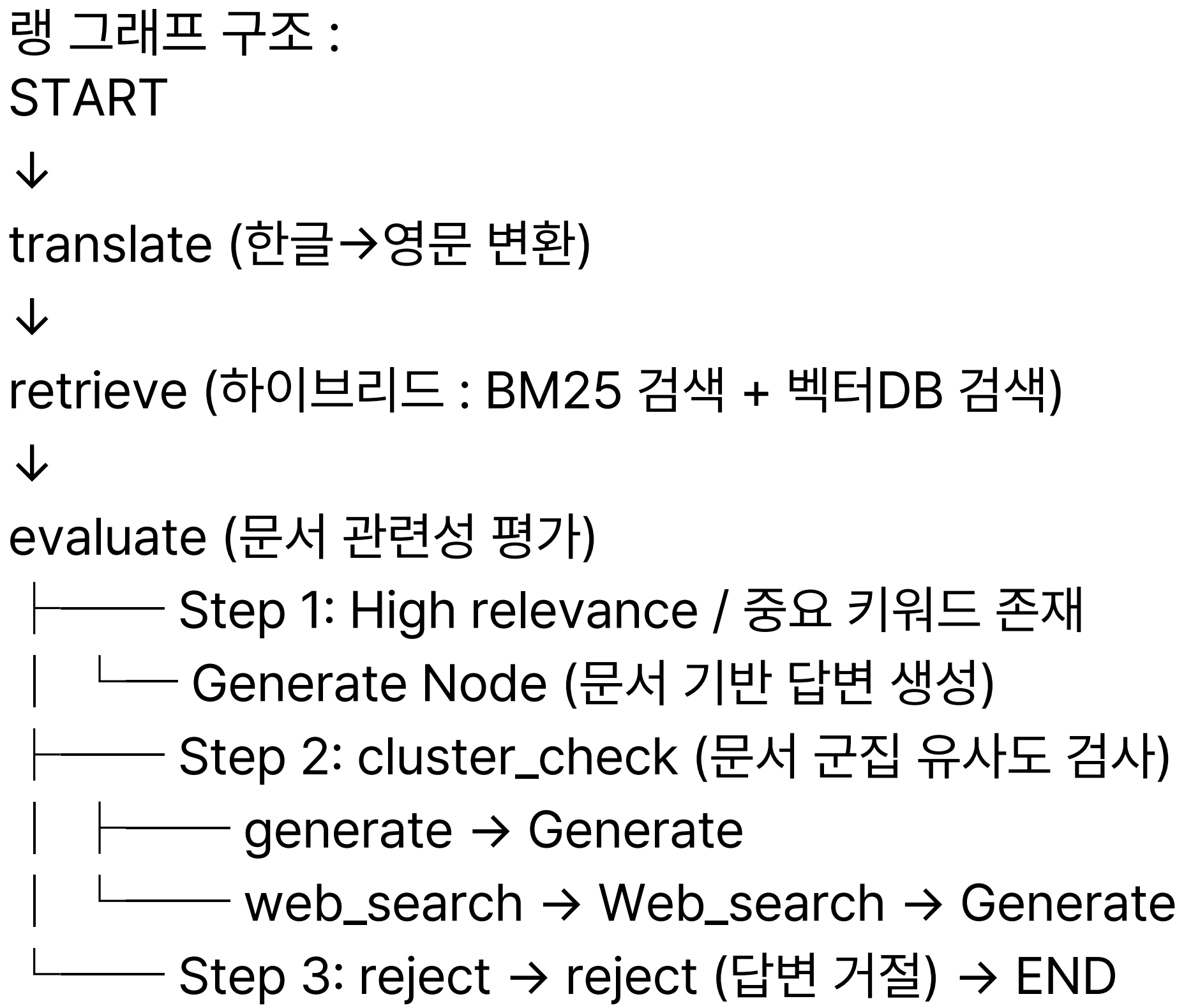
END

개선 랭그래프 구성

+ 하이브리드 검색기능 (BM25 + Dense)

오류 수정 방안 (2) : BM25 사용

- STATE :
- 앞 랭그래프와 STATE 구성 동일
- NODE :
- retriever 노드 : 하이브리드 검색
 - 벡터 검색
 - BM25 검색
 - RRF 점수 계산
 - grade 노드
 - generate 노드
 - translate 노드
 - evaluate 노드
 - cluster_check 노드



최종 랭그래프 구성
+ 하이브리드 검색 기능
+ 클러스터 기반 검색 기능
+ tavily 웹 검색 기능

STATE :

question: str
original_question: str
translated_question: Optional[str]
is_korean: bool
documents: List[Document]
doc_scores: List[float]
cluster_id: Optional[int]
cluster_similarity_score: Optional[float]
search_type: str
relevance_level: str
answer: str
sources: List[Dict[str, Any]]
is_ai_ml_related: bool
_vectorstore: Any
_llm: Any
_bm25_retriever: Any
_cluster_metadata_path: str

NODE :

Translate Node (한글 → 영문)
AI/ML/DL/LLM Relevance Pre-check Node
Hybrid Search Node
Document Relevance Evaluation Node
Generate Node (문서 기반 답변 생성)
Cluster Search Node
Web Search Node
Reject Node → Generate Node (답변 거부)

최종 랭그래프 구성

- + 하이브리드 검색 기능
- + 클러스터 기반 검색 기능
- + tavily 웹 검색 기능

랭 그래프 구조 :

START



Translate Node (한글 → 영문)



AI/ML/DL/LLM Relevance Pre-check Node (부스트)



Hybrid Search Node

- Vector Store Similarity Search

- BM25 Retriever

- Boost Score 추가



Document Relevance Evaluation Node

- ├── Step 1: High relevance / 점수 높음

- └── Generate Node (문서 기반 답변 생성)

- ├── Step 2: Ambiguous relevance / 점수가 애매

- └── Cluster Search Node

- └── High cluster score → Generate Node (문서 기반 답변 생성)

- └── Low cluster score → Web Search Node → Generate Node (웹 기반 답변 생성)

- └── Step 3: No document / 점수 낮음

- └── Step 4: Reject Node → Generate Node (답변 거부) → END

3) html 시험

4. 성능 비교 - 랭스미스_초반

simple-rag-system-cc95c4cf

SKN20-.../0df251

↓

Compact

Full

Diff

Default

Group by

	Inputs	Reference Outputs	Outputs
1.	ToolOrchestra: Elevating I... #359d →	title : ToolOrchestra: Elevating Intellige...	1) 간단한 요약 ToolOrchestra는 다양한 지능형 도구를 효율적으로 조정하여 복잡한 작업을 해결하는 방법을 제시합니다....
2.	LLM에서 캐시와 관련된 논문... #72c6 →	title: Cache-to-Cache: Direct Semantic...	1) LLM에서 캐시와 관련된 논문은 없습니다. 2) 현재 제공된 논문들 중에서 LLM의 캐시와 직접적으로 관련된 내용을 다룬...
3.	LLM에서 긴 문맥의 추론을 향... #8112 →	title: Beyond Fact Retrieval: Episodic M...	1) GSW(Generative Semantic Workspace)는 LLM의 긴 문맥 추론을 향상시키는 새로운 접근법입니다. 2) - GSW는 ...
4.	LLM에서 환각탐지를 할 수 있... #88f6 →	title: When Models Lie, We Learn: Multi...	1) LLM에서 환각 탐지를 위한 데이터셋에 대한 정보입니다. 2) 최근 연구에서는 LLM의 환각 탐지를 위한 다양한 방법과 ...
5.	core attention disaggrega... #8dcb →	title : Efficient Long-context Language ...	1) 핵심 주의 분산(core attention disaggregation, CAD)은 긴 문맥의 대형 언어 모델 훈련을 개선하는 기술입니다. 2) ...
6.	Adrian Kosowski 저자의 최... #9214 →	title: The Dragon Hatchling: The Missin...	NO_RELEVANT_PAPERS [WebResult 1] title: Adrian Kosowski - Google Scholar url: https://scholar.google....
7.	GUI-360: A Comprehensi... #9f83 →	title: GUI-360: A Comprehensive Datas...	1) GUI-360은 컴퓨터 사용 에이전트를 위한 대규모 데이터셋 및 벤치마크입니다. 2) - GUI-360은 실제 작업에서의 데이...
8.	RFT를 LVLMs (large video... #a7ad →	title: VIDEOP2R: Video Understanding f...	1) RFT를 대형 비디오 언어 모델(LVLMs)로 확장하는 것은 도전적입니다. 2) - RFT를 LVLMs에 적용하는 것은 여러 기술...
9.	해리포터 줄거리 알려주세요 #cd5e →	해당없다	NO_RELEVANT_PAPERS [WebResult 1] title: 해리포터 줄거리 url: https://example.com/harry-potter-summar...
10.	최근 공개된 논문에서 좋아요... #eafa →		NO_RELEVANT_PAPERS [WebResult 1] title: Recent Advances in AI and Machine Learning url: https://exa...
11.	오디오 기반 애니메이션의 정... #fd3c →	title: https://huggingface.co/papers/25...	1) 오디오 기반 애니메이션에서 캐릭터 정체성을 유지하는 방법에 대한 연구입니다. 2) - Lookahead Anchoring 기법을 ...

4. 성능 비교 - 랭스미스_최종

langgraph-rag-v2-90ff01f4

SKN20-.../cc879b

Compact

Full

Diff

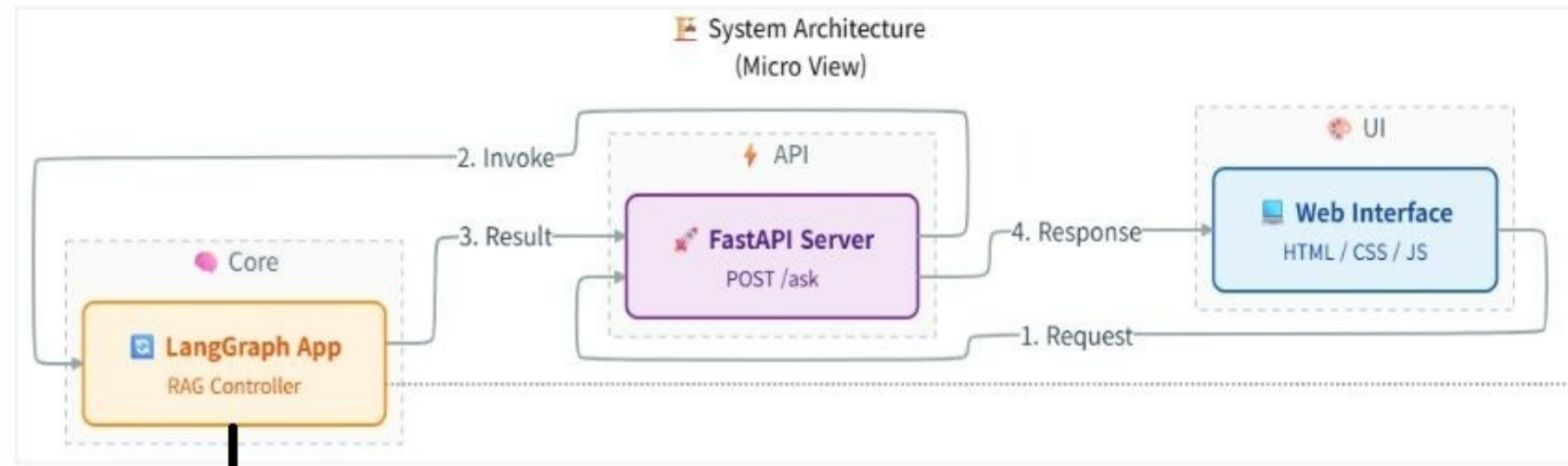
Default

Group by

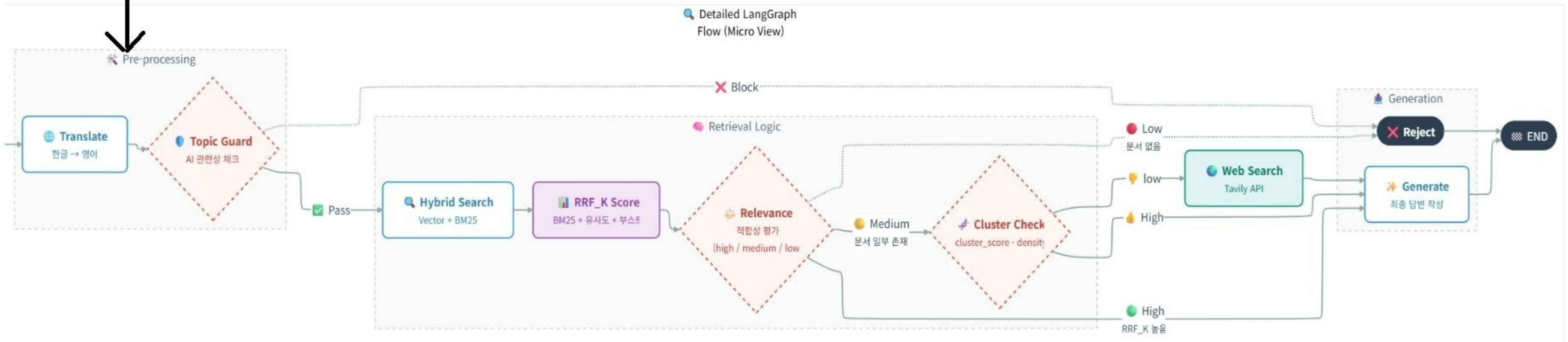
Display

	Inputs	Reference Outputs	Outputs
1	Adrian Kosowski 저자의 최근 논문이 있나요? #04b4 →	title: The Dragon Hatchling: The Missing Li...	죄송합니다. "Adrian Kosowski 저자의 최근 논문이 있나요?"는 AI/ML/DL/LLM 연구 논문과 관련이 없는 질문입니다.
2	GUI-360: A Comprehensive Dataset and Bench... #0690 →	title: GUI-360: A Comprehensive Dataset a...	GUI-360은 컴퓨터 사용 에이전트를 위한 대규모 데이터셋 및 벤치마크로, 실제 작업의 격차, 자동 데이터 수집 및 GUI 기반 평가의 통...
3	LLM에서 환각탐지를 할 수 있는 데이터셋을 알려주세요 #2224 →	title: When Models Lie, We Learn: Multiling...	LLM에서 환각 탐지를 위한 데이터셋으로는 "PsiloQA"가 있습니다. 이 데이터셋은 14개 언어에서 스펙 수준의 환각을 탐지할 수 있도...
4	오디오 기반 애니메이션의 정체성을 유지하는 방법이 ... #2e7b →	title: https://huggingface.co/papers/2510....	죄송합니다. "오디오 기반 애니메이션의 정체성을 유지하는 방법이 있나요?"는 AI/ML/DL/LLM 연구 논문과 관련이 없는 질문입니다.
5	RFT를 LVLMs (large video language models) 으... #542e →	title: VIDEOP2R: Video Understanding fro...	RFT(강화 학습 미세 조정)를 대형 비디오 언어 모델(LVLMs)로 확장하는 방법은 다음과 같습니다. Yifan Jiang의 연구에서 제안된 Vi...
6	LLM에서 긴 문맥의 추론을 향상시키는 GSW (Gener... #61bd →	title: Beyond Fact Retrieval: Episodic Mem...	논문 제목: **Beyond Fact Retrieval: Episodic Memory for RAG with Generative Semantic Workspaces** 저자: Shreyas...
7	core attention disaggregation 은 무엇인가요? #7023 →	title : Efficient Long-context Language Mo...	코어 어텐션 분리(core attention disaggregation, CAD)는 긴 컨텍스트의 대형 언어 모델 훈련을 개선하기 위한 기술입니다. 이 방...
8	해리포터 줄거리 알려주세요 #8039 →	해당없다	죄송합니다. "해리포터 줄거리 알려주세요"는 AI/ML/DL/LLM 연구 논문과 관련이 없는 질문입니다.
9	최근 공개된 논문에서 좋아요수를 300개 이상 받은 논문... #9ffc →		죄송합니다. "최근 공개된 논문에서 좋아요수를 300개 이상 받은 논문은 몇개인가요? (5개 이하인 경우, 5개 논문에 대해서 소개해주...
10	ToolOrchestra: Elevating Intelligence via Efficie... #d27b →	title : ToolOrchestra: Elevating Intelligence...	ToolOrchestra는 다양한 지능형 도구를 조정하는 소형 오케스트레이터를 훈련하는 방법을 제안합니다. 이 방법은 결과, 효율성 및 사...
11	LLM에서 캐시와 관련된 논문이 있나요? #ef32 →	title: Cache-to-Cache: Direct Semantic Co...	LLM(대형 언어 모델)에서 캐시와 관련된 두 가지 주요 논문이 있습니다. 1. **Attention Is All You Need for KV Cache in Diffusi...

5. 최종 시스템 흐름 다이어그램



랭그래프 내부구조



출처

허깅페이스 트렌딩 페이퍼 :

<https://huggingface.co/papers/trending>

청킹 전략화 참고 문서 :

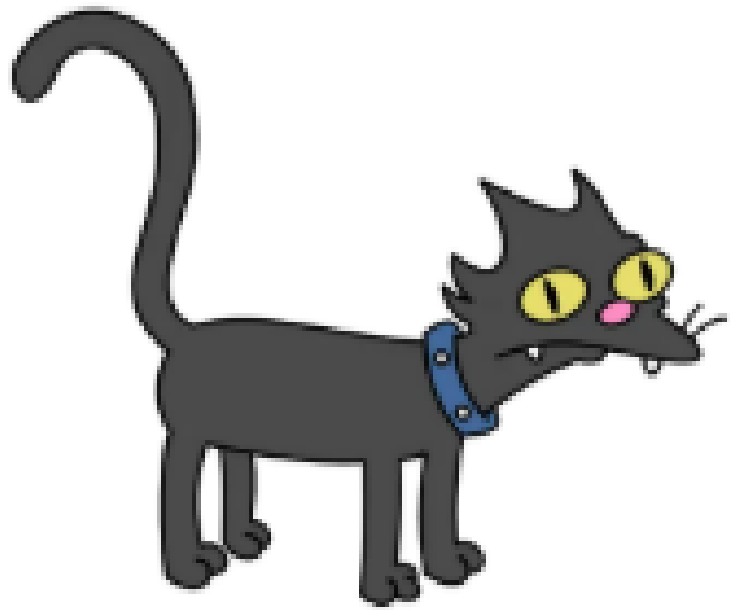
출처: [kt cloud 기술 블로그]

<https://tech.ktcloud.com/entry/2025-11-ktcloud-rag-ai-청킹전략-최적화>

TEAM 해줘.

팀원 소개란

팀원



김지은



박다정



오학성



정소영



황수현

감사합니다.